

Practical Assignments: 1

Set A

- (1) Write a program that demonstrates the use of `nice()` system call. After a child process is started using `fork()`, assign higher priority to the child using `nice()` system call.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>
int main()
{
    pid_t pid;
    int status;
    printf("Parent process started.\n");
    printf("Parent PID: %d\n", getpid());
    printf("Parent nice value: %d\n", nice(0));

    pid = fork();
    if (pid < 0)
    {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }
    if (pid == 0)
    {
        /* Child process */
        int old_nice, new_nice;
        printf("\nChild process started.\n");
        printf("Child PID: %d\n", getpid());
        errno = 0;
        old_nice = nice(0);
        if (errno != 0)
        {
            perror("nice");
            exit(EXIT_FAILURE);
        }
        printf("Child's original nice value: %d\n", old_nice);
        errno = 0;
        new_nice = nice(-5);

        if (errno != 0)
```

```
{
    perror("nice(-5) failed");
    printf("Run the program with appropriate privileges.\n");
    exit(EXIT_FAILURE);
}

printf("Child's new nice value: %d\n", new_nice);
printf("Child now has higher CPU scheduling priority.\n");

exit(EXIT_SUCCESS);
}
else
{
    /* Parent process */
    printf("\nParent waiting for child...\n");

    waitpid(pid, &status, 0);

    printf("Child process completed.\n");
    printf("Parent process completed.\n");
}

return 0;
}
```